



School:Campus:

Academic Year: Subject Name: Subject Code:

Semester: Program: Branch: Specialization:

Date:

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiment: Hello Solidity – Writing First Smart Contract

*Coding Phase: Pseudo Code / Flow Chart / Algorithm

Step 1: Open a web browser and navigate to the Remix IDE at <https://remix.ethereum.org>. From the left sidebar, select the “Solidity” environment to start developing a smart contract.

Step 2: In the Remix file explorer, click “Create New File”, name it SimpleStorage.sol, and press Enter to create the file.

Step 3: Write the Solidity smart contract code inside the SimpleStorage.sol file.
(This contract will typically include simple set and get functions to store and retrieve data.)

Step 4: After completing the code, navigate to the “Solidity Compiler” tab on the left panel and click “Compile SimpleStorage.sol”
Ensure the compilation completes successfully without any errors.

Step 5: Next, go to the “Deploy & Run Transactions” tab.
Under the Environment dropdown, select “Injected Provider – MetaMask” to connect Remix with your MetaMask wallet.

Step 6: Click “Deploy” to deploy the smart contract.
MetaMask will open a popup window — review the transaction details and click “Confirm” to broadcast the transaction to the Sepolia Test Network.

Step 7: After deployment, open MetaMask → Activity tab to view the latest transaction details.
Click “Blockchain Explorer” to view complete transaction information such as transaction ID, gas used, and block confirmation.

Step 8: Back in Remix, under the Deployed Contracts section, you will see your deployed contract instance.
You can now use the set function to store data and the get function to retrieve data — each interaction will reflect on your MetaMask wallet.

Step 9: Once all interactions are complete, the deployment and testing process is successfully finished.
End.

*Software Used

- Laptop
- Remix IDE
- MetaMask browser extension
- Sepolia Etherscan explorer

Page No.....

*** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.**

* Testing Phase: Compilation of Code (error detection)

No error

* Implementation Phase: Final Output (no error)

The collage consists of six screenshots arranged in a 2x3 grid, illustrating the workflow from code compilation to final deployment and interaction.

- Top Left:** A screenshot of the Remix IDE File Explorer and the Solidity Compiler window. The code for `SimpleStorage.sol` is shown, including a `set` function to store a number and a `get` function to retrieve it. The compiler version is 0.8.30+commit.73712a01.
- Top Middle:** A screenshot of the Remix IDE 'DEPLOY & RUN TRANSACTIONS' panel. It shows the environment set to 'Injected Provider - MetaMask' on the 'Sepolia (11155111) network'. The account address is `0xc00...016C7` with a balance of 0.2249261056 ETH. The gas limit is set to 3,000,000. The contract to be deployed is 'SimpleStorage - SimpleStorage.sol'.
- Top Right:** A screenshot of the MetaMask mobile app showing the 'Deploy a contract' screen. It displays 'Estimated changes: No changes' and 'Request from: remix.ethereum.org'. The network fee is 0.0002 SepoliaETH. The speed is set to 'Market ~12 sec'.
- Bottom Left:** A screenshot of the Remix IDE 'DEPLOY & RUN TRANSACTIONS' panel after deployment. It shows the 'SimpleStorage' contract deployed at address `0x238F...aC7Cd`. The 'Deployed Contracts' section shows the contract's state with a balance of 0 ETH. The 'set' function is highlighted with a value of 10.
- Bottom Middle:** A screenshot of the MetaMask mobile app showing the 'Transaction request' screen. It displays 'Estimated changes: No changes' and 'Request from: remix.ethereum.org'. The network fee is 0.0001 SepoliaETH. The speed is set to 'Market ~12 sec'.
- Bottom Right:** A screenshot of the MetaMask mobile app showing the account balance of 0.227 SepoliaETH. The 'Activity' tab shows a 'Contract deployment' transaction for -0 SepoliaETH, which is confirmed.

Page No.....

* As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.

* Observations

The **SimpleStorage** smart contract successfully demonstrates the fundamental concept of smart contracts on the Ethereum blockchain — the ability to **store and retrieve a single unsigned integer (uint256)**.

The **set ()** function updates the stored value, showcasing how **state-changing transactions** work in Solidity and how such operations **consume gas** when executed on the blockchain.

The **get ()** function retrieves the stored value, demonstrating the use of **view functions**, which **do not modify the contract state** and therefore **do not incur any gas cost**.

Testing the contract using **Remix IDE** provided a clear understanding of the **deployment process**, **MetaMask interaction**, and the **execution of smart contract functions** within a blockchain environment.

ASSESSMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	50		

Signature of the Student :

Name :

Signature of the Faculty :

Regn. No. :

Page No.....

*** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used**